

DSA Assignment 520: 20260520模拟考

2500010774 兰玉琪 数学科学学院

1. 题目

E04080: Huffman编码树

<http://cs101.openjudge.cn/practice/04080/>

思路：

代码：

```

import heapq
n = int(input())
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.depth = 0
    def __lt__(self, other):
        return self.val < other.val
mylist = list(map(int, input().split()))
leaves = [TreeNode(mylist[i]) for i in range(n)]
tree = []
for node in leaves:
    tree.append(node)
heapq.heapify(tree)
while len(tree) != 1:
    node1 = heapq.heappop(tree)
    node2 = heapq.heappop(tree)
    node = TreeNode(node1.val+node2.val)
    node.left = node1
    node.right = node2
    heapq.heappush(tree, node)
def dfs(node, depth):
    if node:
        node.depth = depth
        if node.left:
            dfs(node.left, depth+1)
        if node.right:
            dfs(node.right, depth+1)
dfs(tree[0], 0)
total = 0
for node in leaves:
    total += node.val * node.depth
print(total)

```

状态: Accepted

Source Code

```
import heapq
n = int(input())
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
        self.depth = 0
    def __lt__(self, other):
        return self.val < other.val
mylist = list(map(int, input().split()))
leaves = [TreeNode(mylist[i]) for i in range(n)]
tree = []
for node in leaves:
    tree.append(node)
heapq.heapify(tree)
while len(tree) != 1:
    node1 = heapq.heappop(tree)
    node2 = heapq.heappop(tree)
    node = TreeNode(node1.val+node2.val)
    node.left = node1
    node.right = node2
```

基本信息

#: 52590368
题目: E04080
提交人: 25n2500010774(sleepy(25n2500010774))
内存: 3684kB
时间: 29ms
语言: Python3
提交时间: 2026-05-20 15:19:09

M05443: 兔子与樱花

dijkstra, Floyd-Warshall, <http://cs101.openjudge.cn/practice/05443/>

思路:

代码:

```

import heapq
n = int(input())
locs = []
loc_id = {}
for i in range(n):
    loc = input()
    locs.append(loc)
    loc_id[loc] = i
q = int(input())
graph = [[] for _ in range(n)]
distance = [[0]*n for _ in range(n)]
for i in range(q):
    a,b,w = input().split()
    a = loc_id[a]
    b = loc_id[b]
    w = int(w)
    graph[a].append((b,w))
    graph[b].append((a,w))
    distance[a][b] = w
    distance[b][a] = w
def solve(loc1,loc2):
    i1 = loc_id[loc1]
    i2 = loc_id[loc2]
    path = []
    if i1 == i2:
        return [loc1]
    dist = [float("inf")] * n
    dist[i1] = 0
    pq = [(0,i1)]
    prev = {i1:None}
    while pq:
        cur_dist, node = heapq.heappop(pq)
        if node == i2:
            break
        if cur_dist > dist[node]:
            continue
        for nei, weight in graph[node]:
            new_dist = cur_dist + weight
            if new_dist < dist[nei]:
                dist[nei] = new_dist
                heapq.heappush(pq, (new_dist,nei))
                prev[nei] = node
    cur = i2

```

```

while cur or cur == 0:
    path.append(locs[cur])
    if prev[cur] or prev[cur] == 0:
        path.append("(" + str(distance[cur][prev[cur]]) + ")")
    cur = prev[cur]
path.reverse()
return path
r = int(input())
for _ in range(r):
    loc1, loc2 = input().split()
    path = solve(loc1, loc2)
    print("->".join(path))

```

#52590862提交状态

[View](#) [Submit](#) [Statistics](#) [Clarify](#)

状态: **Accepted**

Source Code

```

import heapq
n = int(input())
locs = []
loc_id = {}
for i in range(n):
    loc = input()
    locs.append(loc)
    loc_id[loc] = i
q = int(input())
graph = [[] for _ in range(n)]
distance = [[0]*n for _ in range(n)]
for i in range(q):
    a,b,w = input().split()
    a = loc_id[a]
    b = loc_id[b]
    w = int(w)
    graph[a].append((b,w))
    graph[b].append((a,w))
    distance[a][b] = w
    distance[b][a] = w
def solve(loc1,loc2):
    # bfs
    loc1 = loc_id[loc1]
    loc2 = loc_id[loc2]
    path = []
    cur = loc1
    prev = 0
    while cur or cur == 0:
        path.append(locs[cur])
        if prev[cur] or prev[cur] == 0:
            path.append("(" + str(distance[cur][prev[cur]]) + ")")
        cur = prev[cur]
    path.reverse()
    return path
r = int(input())
for _ in range(r):
    loc1, loc2 = input().split()
    path = solve(loc1, loc2)
    print("->".join(path))

```

基本信息

#: 52590862
 题目: M05443
 提交人: 25n2500010774(sleepy(25n2500010774))
 内存: 3656kB
 时间: 22ms
 语言: Python3
 提交时间: 2026-05-20 15:45:15

M20741: 两座孤岛最短距离

bfs, <http://cs101.openjudge.cn/practice/20741/>

思路:

代码:

```

from collections import deque
n = int(input())
matrix = []
for _ in range(n):
    matrix.append(list(input()))
visited = [[False]*n for _ in range(n)]
vectors = [(0,1),(1,0),(-1,0),(0,-1)]
fenzhi = []
for t in range(n):
    for s in range(n):
        pos = (t,s)
        if visited[t][s] or matrix[t][s] == "0":
            continue
        res = set()
        def dfs(pos):
            a,b = pos[0],pos[1]
            if visited[a][b]:
                return
            res.add(pos)
            visited[a][b] = True
            for i,j in vectors:
                if a+i >= 0 and a+i <= n-1 and b+j >= 0 and b+j <= n-1 and not visited[
                    dfs((a+i,b+j))
            dfs(pos)
        fenzhi.append(res)
visited = [[False]*n for _ in range(n)]
q = deque()
for pos in fenzhi[0]:
    q.append((pos,0))
    visited[pos[0]][pos[1]] = True
while q:
    cur,step = q.popleft()
    if cur in fenzhi[1]:
        print(step-1)
        break
    a = cur[0]
    b = cur[1]
    for i,j in vectors:
        if a+i >= 0 and a+i <= n-1 and b+j >= 0 and b+j <= n-1 and not visited[i+a][j+b]
            q.append(((a+i,j+b),step+1))
            visited[a+i][j+b] = True

```

状态: **Accepted**

Source Code

```
from collections import deque
n = int(input())
matrix = []
for _ in range(n):
    matrix.append(list(input()))
visited = [[False]*n for _ in range(n)]
vectors = [(0,1), (1,0), (-1,0), (0,-1)]
fenzhi = []
for t in range(n):
    for s in range(n):
        pos = (t,s)
        if visited[t][s] or matrix[t][s] == "0":
            continue
        res = set()
        def dfs(pos):
            a,b = pos[0],pos[1]
            if visited[a][b]:
                return
            res.add(pos)
            visited[a][b] = True
            for i,j in vectors:
                if a+i >= 0 and a+i <= n-1 and b+j >= 0 and b+j <= n-1 and matrix[a+i][b+j] != "0":
                    dfs((a+i,b+j))
```

基本信息

#: 52591168

题目: M20741

提交人: 25n2500010774(sleepy(25n2500010774))

内存: 4724kB

时间: 40ms

语言: Python3

提交时间: 2026-05-20 15:59:39

M24637: 宝藏二叉树

dp, dfs <http://cs101.openjudge.cn/practice/24637/>

思路:

代码:

```

class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None

n = int(input())
mylist = list(map(int, input().split()))

def fun(i):
    node = TreeNode(mylist[i])
    if 2*i+1 < n:
        node.left = fun(2*i+1)
    if 2*i + 2 < n:
        node.right = fun(2*i+2)
    return node

root = fun(0)

def dfs(node):
    total = 0
    if not node:
        return 0
    total = max(total, dfs(node.left)+dfs(node.right))
    cur = node.val
    if node.left:
        cur += dfs(node.left.left) + dfs(node.left.right)
    if node.right:
        cur += dfs(node.right.left) + dfs(node.right.right)
    total = max(total, cur)
    return total

print(dfs(root))

```


状态: **Accepted**

Source Code

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
n = int(input())
mylist = list(map(int, input().split()))
def fun(i):
    node = TreeNode(mylist[i])
    if 2*i+1 < n:
        node.left = fun(2*i+1)
    if 2*i + 2 < n:
        node.right = fun(2*i+2)
    return node
root = fun(0)
def dfs(node):
    total = 0
    if not node:
        return 0
    total = max(total, dfs(node.left)+dfs(node.right))
    cur = node.val
    if node.left:
        cur += dfs(node.left.left) + dfs(node.left.right)
    if node.right:
        cur += dfs(node.right.left) + dfs(node.right.right)
    total = max(total, cur)
```

基本信息

#: 52591308
题目: M24637
提交人:
25n2500010774(sleepy(25n2500010774))
内存: 3728kB
时间: 44ms
语言: Python3
提交时间: 2026-05-20 16:10:08

T02337: Catenyms

Eulerian Path, <http://cs101.openjudge.cn/practice/02337/>

思路:

让gpt一点点才把我教会,

觉得很神奇, 这个dfs每次选择路径那么随机没想到竟然在回溯后添加路径是唯一的

代码

```
import sys
sys.setrecursionlimit(100000)

input = sys.stdin.readline

T = int(input())

for _ in range(T):

    n = int(input())

    # graph[u]:
    # 存 (单词,v)
    graph = [[] for _ in range(26)]

    indeg = [0] * 26
    outdeg = [0] * 26

    # 建图
    for _ in range(n):

        word = input().strip()

        u = ord(word[0]) - ord('a')
        v = ord(word[-1]) - ord('a')

        graph[u].append((word,v))

        outdeg[u] += 1
        indeg[v] += 1

    # 为字典序最小
    for i in range(26):
        graph[i].sort(reverse=True)

    # 找欧拉路径起点
    start = -1
    end = -1

    ok = True

    for i in range(26):
```

```

diff = outdeg[i] - indeg[i]

if diff == 1:

    if start == -1:
        start = i
    else:
        ok = False

elif diff == -1:

    if end == -1:
        end = i
    else:
        ok = False

elif diff != 0:
    ok = False

if not ok:
    print("***")
    continue

# 欧拉回路情况
if start == -1:

    for i in range(26):
        if outdeg[i]:
            start = i
            break

ans = []

# Hierholzer
def dfs(u):

    while graph[u]:

        word, v = graph[u].pop()

        dfs(v)

    ans.append(word)

```

```

dfs(start)

# 必须所有边都经过
if len(ans) != n:
    print("***")
    continue

ans.reverse()

print(".".join(ans))

```

#52621276提交状态

View Submit Statistics Clarify

状态: Accepted

Source Code

```

import sys
sys.setrecursionlimit(100000)

input = sys.stdin.readline

T = int(input())

for _ in range(T):
    n = int(input())

    # graph[u]:
    # 存 (单词,v)
    graph = [[] for _ in range(26)]

    indeg = [0] * 26
    outdeg = [0] * 26

    # 建图
    for _ in range(n):
        word = input().strip()

        u = ord(word[0]) - ord('a')
        v = ord(word[-1]) - ord('a')

        graph[u].append((word,v))

        outdeg[u] += 1
        indeg[v] += 1

```

基本信息

#: 52621276
 题目: 02337
 提交人: 25n2500010774(sleepy(25n2500010774))
 内存: 4524kB
 时间: 23ms
 语言: Python3
 提交时间: 2026-05-24 16:02:35

T30878:力场叠加模拟

segment tree, lazy propagation, <http://cs101.openjudge.cn/practice/30878/>

思路:

让gpt一点点才把我教会，还有就是用debug的功能自己想了一遍

代码

```

import sys
sys.setrecursionlimit(1 << 25)

def main():
    import sys
    input = sys.stdin.read
    data = input().split()
    idx = 0
    N = int(data[idx])
    idx += 1
    Q = int(data[idx])
    idx += 1

    # 线段树数组 & 懒标记
    size = 1
    while size < N:
        size <= 1
    tree = [0] * (2 * size)
    lazy = [0] * (2 * size)

    # 向下传递懒标记
    def push_down(node, l, r):
        if lazy[node] != 0 and node < size:
            # 左孩子
            tree[2*node] += lazy[node]
            lazy[2*node] += lazy[node]
            # 右孩子
            tree[2*node+1] += lazy[node]
            lazy[2*node+1] += lazy[node]
            # 清空当前节点懒标记
            lazy[node] = 0

    # 区间加 v
    def update(a, b, v, node=1, l=1, r=None):
        if r is None:
            r = size
        if a > r or b < l:
            return
        if a <= l and r <= b:
            tree[node] += v
            lazy[node] += v
            return
        push_down(node, l, r)

```

```

mid = (l + r) // 2
update(a, b, v, 2*node, l, mid)
update(a, b, v, 2*node+1, mid+1, r)
tree[node] = max(tree[2*node], tree[2*node+1])

```

区间查询最大值

```

def query(a, b, node=1, l=1, r=None):
    if r is None:
        r = size
    if a > r or b < l:
        return -float('inf')
    if a <= l and r <= b:
        return tree[node]
    push_down(node, l, r)
    mid = (l + r) // 2
    left = query(a, b, 2*node, l, mid)
    right = query(a, b, 2*node+1, mid+1, r)
    return max(left, right)

```

处理询问

```

output = []
for _ in range(Q):
    op = data[idx]
    idx += 1
    l = int(data[idx])
    idx += 1
    r = int(data[idx])
    idx += 1
    if op == 'Add':
        v = int(data[idx])
        idx += 1
        update(l, r, v)
    else:
        res = query(l, r)
        output.append(str(res))

```

```

print('\n'.join(output))

```

```

if __name__ == "__main__":
    main()

```

状态: **Accepted**

Source Code

```
import sys
sys.setrecursionlimit(1 << 25)

def main():
    import sys
    input = sys.stdin.read
    data = input().split()
    idx = 0
    N = int(data[idx])
    idx += 1
    Q = int(data[idx])
    idx += 1

    # 线段树数组 & 懒标记
    size = 1
    while size < N:
        size <= 1
    tree = [0] * (2 * size)
    lazy = [0] * (2 * size)

    # 向下传递懒标记
    def push_down(node, l, r):
        if lazy[node] != 0 and node < size:
            # 左孩子
            tree[2*node] += lazy[node]
            lazy[2*node] += lazy[node]
            # 右孩子
            tree[2*node+1] += lazy[node]
            lazy[2*node+1] += lazy[node]
```

基本信息

#: 52621119
 题目: 30878
 提交人: 25n2500010774(sleepy(25n2500010774))
 内存: 48560kB
 时间: 4217ms
 语言: Python3
 提交时间: 2026-05-24 15:54:28

2. 学习总结和个人收获

只能ac4了www，期末感觉一片灰暗啊，做了一年往年题(cs201 2025spring Final Exam)，只能ac5，那个神经网络那道题真的就是文字游戏，语文太差了